

ChéraSakhi

CHERANMOOLA LADIES WING

Software Architecture Blueprint

Kuri Ledger · Pamparam Lucky Draw · Tour & Party Wallet
Flutter · Riverpod · Firebase — complete technical design

Document	Codebase	Date	Prepared for
Architecture Blueprint v1.0	kuri_collective 1.0.0+1	October 2026	Sabeer

BLUEPRINT

Contents

1	Executive summary	12	State management (Riverpod)
2	System context	13	Security model
3	Technology stack	14	Offline-first & sync
4	Clean architecture layers	15	Screens & navigation
5	Project structure	16	Design system & brand
6	Users, roles & permissions	17	Integrations
7	Authentication flow	18	Testing strategy
8	Data model & Firestore schema	19	Setup, build & deployment
9	Module 1 — Kuri Ledger	20	Operations runbook
10	Module 2 — Pamparam Lucky Draw	21	Limitations & roadmap
11	Module 3 — Tour & Party Wallet	A	Appendix — source file inventory

How to read this document

This blueprint describes the CheraSakhi mobile application exactly as it exists in the delivered source package (`cherasakhi_app.zip`). Every module, collection, rule and screen described here maps to a real file, listed in the appendix. Diagrams use the app's own palette: **berry** for client-side components, **amber** for Firebase services and **green** for external apps.

3	60	61	6.0k
CORE MODULES	MEMBER CAPACITY	DART SOURCE FILES	LINES OF DART

SECTION 1

Executive summary

CheraSakhi is the official mobile app of the Cheranmoola Ladies Wing, a 60-member neighbourhood women's self-help group. It replaces paper registers and WhatsApp spreadsheets with one shared, tamper-resistant record of monthly savings, the monthly lucky draw and the money spent on trips and celebrations.

The problems it solves

Who has paid this month?
Collectors track 60 members by hand. Disputes arise over missed payments and nobody sees the running total.

Is the draw fair?
Members must trust that only paid members are in the draw and that past winners can't win again.

Where is the trip money?
Cash moves between several women during a trip, and nobody knows who is holding how much.

The three modules

Module	What it does	Key guarantee
Kuri Ledger Monthly chit fund	60-member checklist per month; one-tap Paid/Unpaid; live totals (target pool, collected, deficit, "52/60 paid"); search and filters; WhatsApp reminders to one member or all unpaid; history by year and month.	Works offline; every change syncs automatically.
Pamparam Draw Monthly lucky draw	Animated spinning wheel showing only eligible members; realistic slow-down with tick sounds and vibration; confetti and trophy celebration; shareable winner card for the WhatsApp group.	Only paid members who have never won this cycle can be drawn. One draw per month. Results can never be edited.
Tour & Party Wallet Event funds	Separate fund per trip or celebration; contributions record who physically received the cash; expenses by category with receipt photos and per-head split; cash handovers between members.	Live "who is holding how much cash right now" board that always reconciles.

Architecture at a glance

Client — Flutter 3 app for Android and iOS, Clean Architecture (domain / data / presentation per feature), Riverpod 2 for state, custom design system with glass surfaces and a 3D brand badge.

Backend — Serverless Firebase: Authentication (phone OTP + admin PIN), Cloud Firestore with offline persistence, Cloud Storage for receipts. Business integrity enforced by Firestore Security Rules.

Design principles

- **One source of truth.** Every phone reads the same live Firestore data; no figures are typed twice.
- **Trust by construction.** Fairness rules are enforced on the server, not just hidden in the UI.
- **Offline first.** Collectors can mark payments in a house with no signal; the app syncs later.
- **Simple for every member.** Phone-number login, large tap targets, Malayalam-friendly WhatsApp messages.

SECTION 2

System context

The app has no custom server. Each phone talks directly to Google Firebase over HTTPS, and hands off to WhatsApp and the phone's share sheet for messaging. Firebase Security Rules sit in front of every read and write.

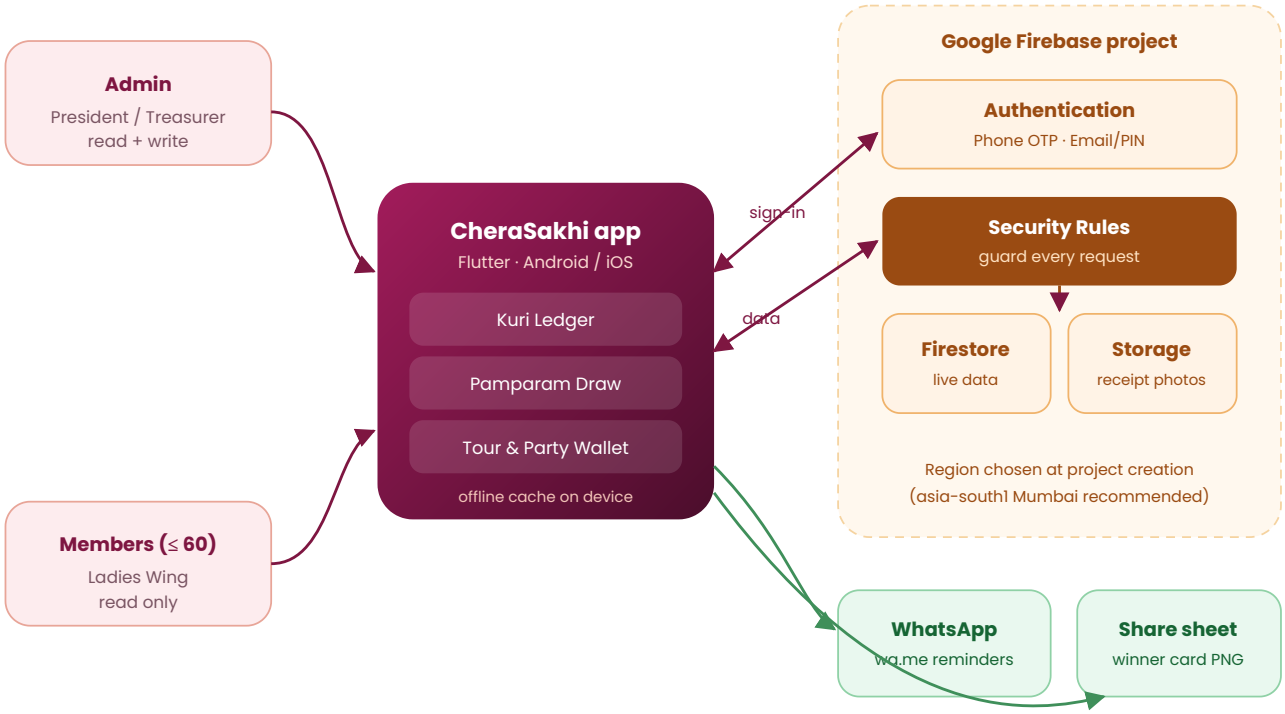


Figure 1 — System context. There is no custom backend server; Firebase Security Rules are the enforcement point.

External dependencies

System	Used for	Direction	Failure behaviour
Firebase Authentication	Phone OTP for members, email + PIN for admins	App → Firebase	Cannot sign in without network the first time; existing sessions persist offline.
Cloud Firestore	All business data, real-time listeners	Two-way	Reads served from local cache; writes queued and synced later.
Cloud Storage	Expense receipt images	App → Firebase	Expense with receipt waits for upload; user sees an error if offline.
WhatsApp	Payment reminders via `wa.me` deep link	App → WhatsApp	Snackbar “WhatsApp is not available”.
OS share sheet	Sharing the winner celebration card	App → OS	Snackbar with error text.

SECTION 3

Technology stack

Layer	Technology	Why it was chosen
Language / UI	Dart 3.4+, Flutter 3.22+, Material 3	One codebase for Android and iOS; 60 fps animations for the wheel; pattern matching and sealed-style `switch` expressions.
State management	`flutter_riverpod` ^2.6.1	Compile-safe providers, automatic disposal, easy composition of Firestore streams; no BuildContext needed for data access.
Authentication	`firebase_auth` ^5.3.1	Phone OTP is natural for members who don't use email; email/password gives admins a PIN fallback.
Database	`cloud_firestore` ^5.4.4	Real-time sync to every phone, built-in offline cache, transactions, declarative security rules.
File storage	`firebase_storage` ^12.3.4	Receipt photos, protected by storage rules that check the user's Firestore role.
Typography	`google_fonts` ^6.2.1	Playfair Display (headings) and Poppins (body) without bundling font files.
Formatting	`intl` ^0.19.0	Indian-rupee currency (en_IN grouping, e.g. ₹1,20,000), month labels.
Celebration	`confetti` ^0.8.0, `audioplayers` ^6.1.0	Confetti bursts and low-latency tick / win sounds.
Sharing	`share_plus` ^10.1.2, `path_provider` ^2.1.4	Export the winner card as PNG and share to WhatsApp.
Deep links	`url_launcher` ^6.3.1	Opens WhatsApp with a pre-filled reminder.
Media	`image_picker` ^1.1.2	Camera / gallery receipts, compressed to quality 70, max width 1600 px.
Tooling	`flutter_lints`, `flutter_launcher_icons`, FlutterFire CLI, Firebase CLI	Lint rules, app icon generation, Firebase project wiring and rule deployment.

Deliberate choice — no code generation. The original brief mentioned Freezed models. Entities were instead written as hand-made immutable classes (const constructors, `copyWith`, `==` / `hashCode`) with mappers in the data layer. The app therefore builds with plain `flutter pub get` and no `build_runner` step, which keeps setup simple for a volunteer maintainer.

Platform targets

Platform	Minimum	Notes
Android	minSdk 23 (Android 6.0)	Required by Firebase Auth. Add SHA-1 and SHA-256 fingerprints and enable Play Integrity for phone auth.

iOS	13.0	APNs key for silent-push phone verification; camera and photo-library usage strings in Info.plist.
Orientation	Portrait only	Locked in `main.dart` for consistent layouts.

SECTION 4

Clean architecture layers

Each feature (`auth` , `kuri` , `lottery_wheel` , `events`) is split into three layers. Dependencies point inward only: presentation depends on domain, data implements domain, and domain depends on nothing outside pure Dart.

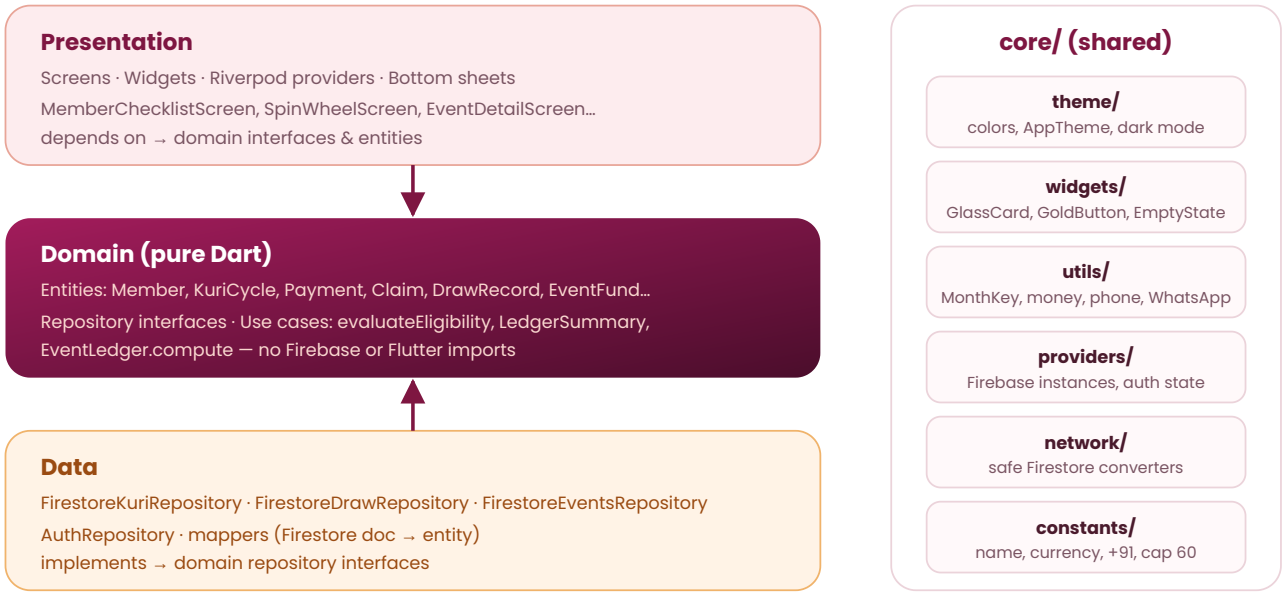


Figure 2 — Layer dependencies. Arrows point toward what a layer depends on.

Responsibilities and rules

Layer	Contains	Must not
Domain	Immutable entities, repository interfaces (`abstract interface class`), pure business logic such as eligibility and ledger maths.	Import Flutter, Firebase or any package with side effects.
Data	Firestore / Storage / Auth implementations, document mappers, transactions, batch writes.	Contain UI logic or hold widget state.
Presentation	Screens, widgets, sheets, Riverpod providers that combine streams into view models (e.g. `WheelData`).	Talk to Firestore directly; it always goes through a repository provider.

Why it matters here: the fairness logic of the lucky draw (`evaluateEligibility`) and the cash-holder maths (`EventLedger.compute`) are pure functions. They are unit-tested without Firebase and could be moved to a Cloud Function later without rewriting them.

SECTION 5

Project structure

```
kuri_collective/
├── pubspec.yaml                # dependencies, assets, launcher-icon config
├── analysis_options.yaml       # lint rules
├── firebase.json               # points CLI at rules + indexes
├── firestore.rules             # database security rules (126 lines)
├── storage.rules               # receipt upload rules
├── firestore.indexes.json
├── README.md                   # setup guide
├── assets/
│   ├── brand/   logo.png · app_icon.png · app_icon_foreground.png
│   └── sounds/  tick.wav · win.wav
├── docs/      firestore_schema.json · logo_build.py
├── test/      widget_test.dart   # eligibility, ledger, cash-holder tests
└── lib/
    ├── main.dart               # Firebase init, offline cache, portrait lock
    ├── app.dart                # MaterialApp, light/dark theme
    ├── firebase_options.dart   # generated by `flutterfire configure`
    ├── core/
    │   ├── constants/  theme/  utils/  network/  providers/  widgets/
    └── features/
        ├── auth/
        │   ├── data/auth_repository.dart
        │   ├── domain/app_user.dart
        │   └── presentation/  providers/ · screens/ (auth_gate, login) · widgets/
        ├── home/presentation/home_shell.dart      # 3-tab bottom navigation
        ├── kuri/
        │   ├── domain/entities/   member · kuri_cycle · payment · claim · ledger_summary
        │   ├── domain/repositories/kuri_repository.dart
        │   ├── data/              firestore_kuri_repository · kuri_mappers
        │   └── presentation/      member_checklist_screen · kuri_history_screen
        │                          widgets: metrics bento, payment tile, month switcher,
        │                                   member form, start-cycle sheet
        ├── lottery_wheel/
        │   ├── domain/            draw_record · draw_repository · usecases/eligibility
        │   ├── data/              firestore_draw_repository   # transaction
        │   └── presentation/      spin_wheel_screen · wheel_painter · wheel_sfx
        │                          winner_celebration
        └── events/
            ├── domain/            event_models · events_repository
            ├── data/              firestore_events_repository  # + Storage
            └── presentation/      events_list_screen · event_detail_screen
                                   event_sheets · member_dropdown · expense_style
```

Naming conventions

- Files `snake_case.dart`; classes `PascalCase`; providers end in `Provider`.
- Firestore implementations are prefixed `Firestore…Repository`; the interface carries the plain name.
- Months are always the sortable string `yyyy-MM` (e.g. `2026-10`) via the `MonthKey` helper.
- Phone numbers are always E.164 (`+919876543210`) via `normalizePhone()`.

SECTION 6

Users, roles & permissions

Admin

President, secretary or treasurer. Signs in with **email + 6-digit PIN** (or a member phone promoted to admin). Can manage members, start cycles, mark payments, spin the wheel and record wallet entries.

Member

Any of the 60 women. Signs in with her **mobile number + OTP**. Sees everything live — the ledger, the wheel spinning, the cash board — but cannot change anything.

Action	Admin	Member	Enforced by
View ledger, history, draws, wallet	Yes	Yes	Rules: `isMember()`
Add / edit / deactivate member	Yes	No	Rules + UI hides menu
Start a Kuri cycle	Yes	No	Rules + UI
Toggle Paid / Unpaid	Yes	No	Rules; members see a PAID/DUE pill instead of a switch
Send WhatsApp reminder	Yes	Yes	Device action only, no data change
Spin the Pamparam wheel	Yes	No	Rules: create-only draws
Edit or delete a past draw / claim	No	No	Rules: `allow update, delete: if false`
Create fund, add contribution / expense / handover	Yes	No	Rules
Upload receipt photo	Yes	No	Storage rules (role lookup)
Change someone's role	Yes	No	Rules; first admin made in console
Share winner card	Yes	Yes	Device action only

Important: hiding a button is never the protection. Every write is checked again by Firestore Security Rules on Google's servers, so a modified app cannot bypass these permissions.

SECTION 7

Authentication flow

`AuthGate` watches the Firebase auth stream and the user's profile document, and decides which screen to show. A new phone user is linked automatically to her member record by matching the verified phone number.

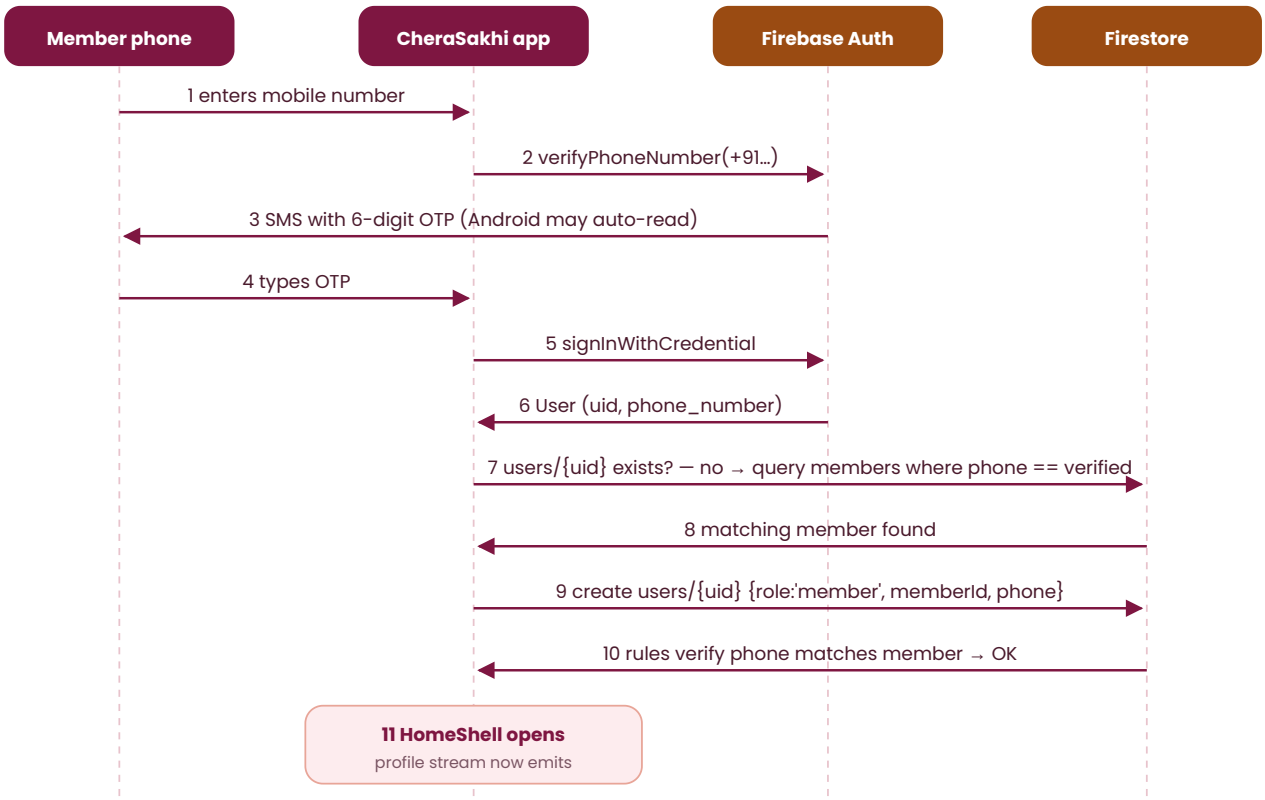


Figure 3 — First phone login and automatic member linking.

AuthGate decision table

Firebase user	Profile users/{uid}	Screen shown
loading	—	Branded splash (3D logo + spinner)
null	—	`LoginScreen` (Member OTP / Admin PIN tabs)
signed in	exists	`HomeShell`
signed in	missing	`_ProfileBootstrap` tries member linking; if no match → "Not registered yet" screen showing the UID to send to the admin, plus Sign out

Admin PIN login

Admins are ordinary Firebase Email/Password accounts whose password is a 6+ digit PIN, created once in the Firebase console. Their `users/{uid}` document is created by hand with `role: "admin"`. Self-registration can only ever create `role: "member"`; the rules reject anything else.

SECTION 8

Data model & Firestore schema

Five top-level collections. Cycle-specific data lives in subcollections of its cycle, and wallet data in subcollections of its event, so funds never mix.

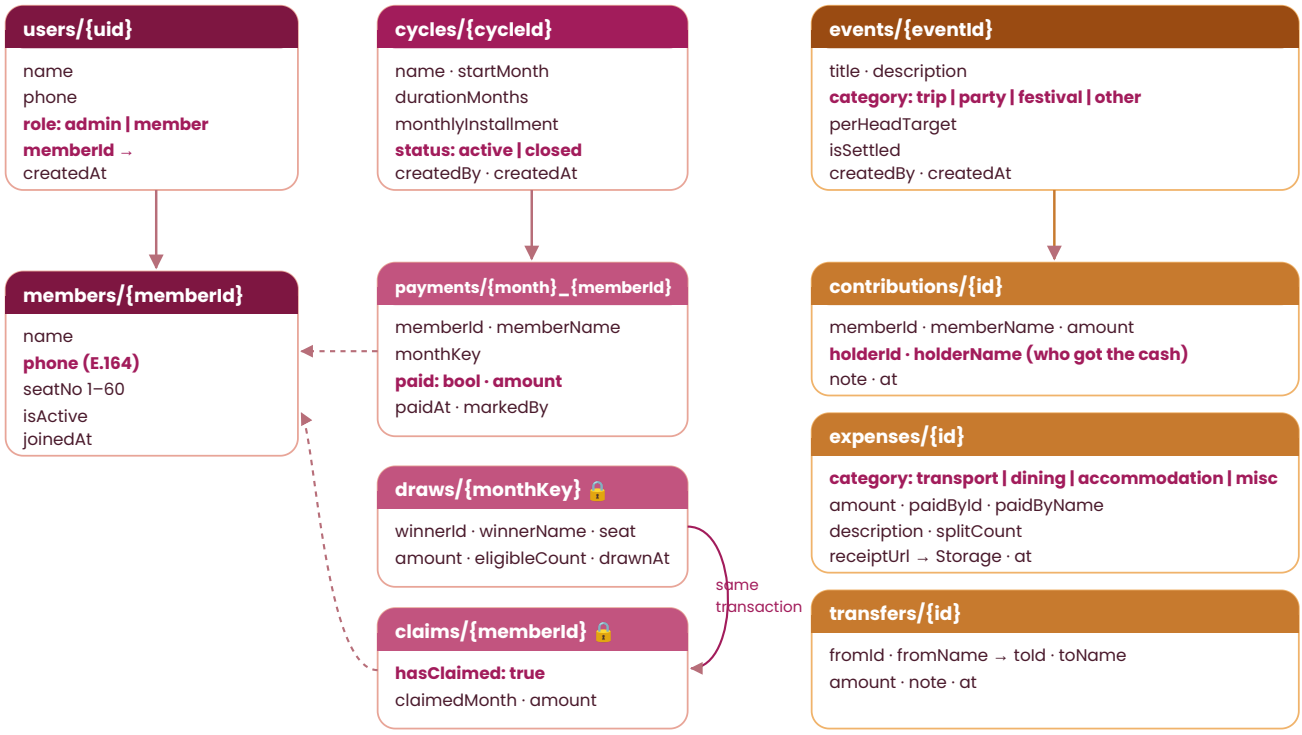


Figure 4 — Entity relationships. 🔒 = create-only (immutable) documents. Dashed arrows = reference to a member.

Key design decisions

Decision	Reason
Payment ID = `{monthKey}_{memberId}`	Exactly one payment record per member per month; toggling overwrites, never duplicates. Rules can find the winner's payment by building the ID.
Draw ID = `{monthKey}`	A second draw for the same month is physically impossible — it would be an update, which rules forbid.
Claim ID = `{memberId}`	A member can hold only one claim per cycle; that is how "never wins twice" is guaranteed.
Names copied into payments, draws, wallet entries	History stays readable even if a member is later renamed or deactivated; avoids extra reads.
Balances are not stored	Totals and cash-holder balances are computed from the entries every time, so they can never drift out of sync.
Only one `active` cycle	Starting a new cycle closes the previous one in the same batch.

The full field-level schema is shipped in `docs/firestore_schema.json`.

SECTION 9

Module 1 — Kuri Ledger

The monthly chit-fund register. Each month every active member pays the same installment; the pot goes to that month's lucky-draw winner.

Features

Admin ledger

- One-tap switch (or tap the row) to mark Paid / Unpaid, with haptic feedback.
- Month switcher bounded to the cycle ("Month 7 of 60").
- Long-press a member to edit name, phone or active status.
- Add member auto-assigns the next seat number; blocks duplicates and more than 60 active.

Everyone

- Bento metrics: Collected (animated), Target Pool, Deficit, Unpaid, progress ring "52/60 Paid".
- Search by name, phone or seat number; filter chips All / Paid / Unpaid.
- Trophy icon next to members who have already won.
- History screen: year chips → month chips → paid / unpaid lists and that month's winner.

Ledger maths (`LedgerSummary.compute`)

```
// only ACTIVE members count
totalMembers = count(active members)
paidCount    = count(active members whose payment.paid == true)
target       = monthlyInstallment × totalMembers
collected    = Σ payment.amount  for paid active members
deficit      = max(0, target − collected)
progress     = collected ÷ target        // drives the gold ring
```

Worked example. 60 members × ₹1,000 = target ₹60,000. 52 have paid → collected ₹52,000, deficit ₹8,000, ring at 87%, label "52/60".

Toggle a payment — sequence

1. Admin taps the switch → `HapticFeedback.lightImpact()`.
2. `setPayment()` writes `cycles/{id}/payments/{month}_{memberId}` with `paid`, amount, `paidAt` server timestamp and `markedBy`.
3. Firestore applies the write to the local cache immediately — the switch, totals and ring update in the same frame, even offline.
4. The write syncs to the server; all other phones receive it through their live listeners.
5. If the server rejects it (e.g. not admin), a snackbar shows "Could not update …".

WhatsApp reminders

Type	Trigger	Message

Individual	Green chat icon on an unpaid member	“Hello {name} 🌸 Gentle reminder from **CheraSakhi**: your Kuri installment of **₹1,000** for **October 2026** is still pending… Please pay before the Pamparam lucky draw so you stay eligible 🎡”
Group	“Remind all unpaid” chip	Numbered list of all unpaid names; WhatsApp opens the chat picker so the admin can post it in the group.

Cycle lifecycle

Admin opens **Start new cycle**: name, monthly installment, start month, number of months (defaults to the active member count). The batch closes any active cycle and creates the new one. Months outside the cycle can't be selected.

SECTION 10

Module 2 — Pamparam Lucky Draw

A live, animated spinning wheel that picks the monthly Kuri winner fairly and transparently, in front of everyone.

Eligibility rule (hard constraint)



Figure 5 — `evaluateEligibility()`: three lists are produced — eligible, unpaid, already won — and all are shown in the Eligibility panel for transparency.

Draw sequence — commit first, animate second

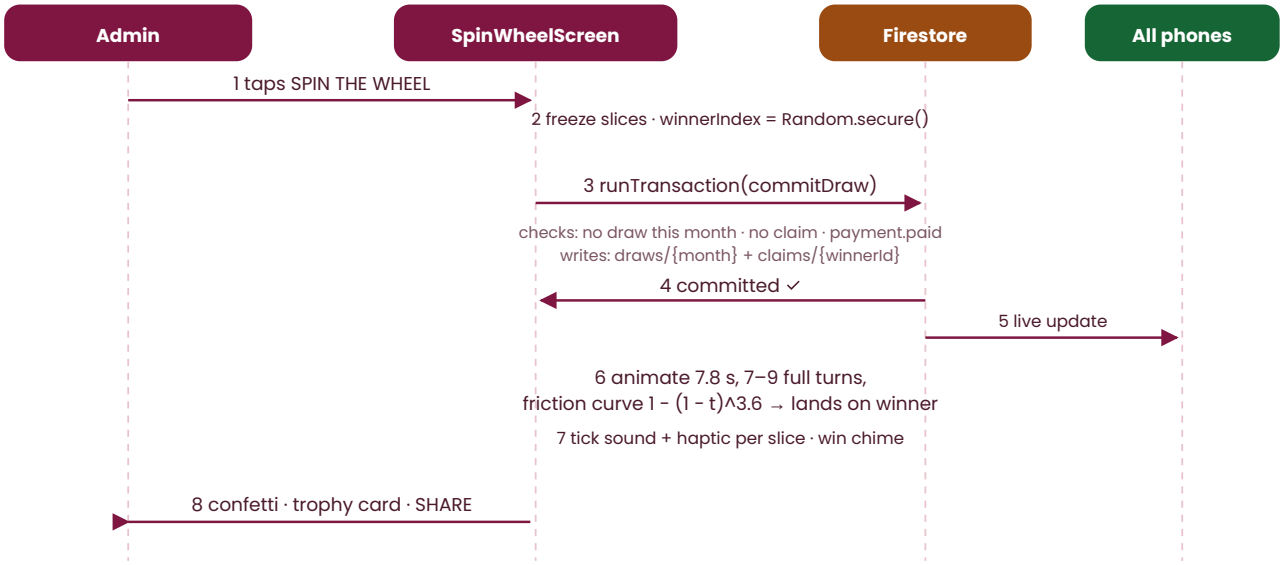


Figure 6 — The winner is saved before the wheel moves. Closing the app mid-spin cannot be used to re-roll.

Wheel geometry

Slices are painted once with `CustomPainter` inside a `RepaintBoundary`; only a `Transform.rotate` changes per frame, keeping the animation at 60 fps even with 60 names. Slice *i* spans angle `−π/2 + i·(2π/n)` to the next slice. The final rotation is chosen so the 12 o'clock pointer rests inside the winner's slice, with a random jitter of ±35 % of the slice width so the landing looks natural.

Detail	Implementation
Colours	Alternating berry and rose-gold slices; an odd final slice is blush so two equal colours never touch.
Label size	17 pt for ≤ 6 names down to 7.5 pt for 40–60 names; above 24 names only the first name is shown.

Sounds	Pool of 4 low-latency players for overlapping ticks (min 40 ms apart); `win.wav` arpeggio on landing.
Celebration	Two confetti emitters (rain + explosive burst) in rose gold, blossom, lavender and white.
Share card	Captured from a `RepaintBoundary` at 3× pixel ratio → PNG → `Share.shareXFiles` with a congratulation message.
Members' view	Same wheel and eligibility panel, but no spin button; the result appears live and the “Draw completed” card offers Celebrate & Share.

SECTION 11

Module 3 — Tour & Party Wallet

Each trip or celebration has its own isolated fund. The headline feature answers the question that causes the most arguments: **who is holding how much cash right now?**

Entry types

Contribution
Member X paid ₹ into the fund; **holder** Y physically received it.

Expense
Category, amount, which holder's cash paid it, description, split count, optional receipt photo.

Cash handover
Holder A gave ₹ to holder B (e.g. before the bus left).

Cash-holder formula (`EventLedger.compute`)

```
for each person P:
  received        = Σ contributions where holder == P
  transferredIn   = Σ handovers      where to     == P
  spent           = Σ expenses       where paidBy == P
  transferredOut  = Σ handovers      where from   == P

  balance(P) = received + transferredIn − spent − transferredOut

  balance > 0  →  "cash in hand"     (green)
  balance < 0  →  "fund owes her"    (red — she paid from her own pocket)

fund balance = Σ contributions − Σ expenses   // = Σ balance(P), always
```

Worked example (matches the unit test).

Entry	Holder H	Holder K
A pays ₹500 to H	+500	
B pays ₹500 to H	+500	
H hands ₹200 to K	-200	+200
K pays lunch ₹300 (split 2)		-300
Balance	**₹800 in hand**	**fund owes ₹100**

Fund balance = 1,000 - 300 = **₹700** = 800 + (-100). ✓

Event detail screen

- **Summary bento:** fund balance (rose-gold hero), collected, spent, number of holders, per-head target and contributors.
- **Holder board:** each holder with received / paid totals and live balance, sorted highest first.
- **Spending by category:** horizontal bars for Transport, Dining, Accommodation, Misc.

- **Timelines:** expenses (tap to view receipt full-screen with zoom), contributions, handovers.

Receipts

Photos are compressed on the phone, uploaded to `events/{eventId}/receipts/{expenseId}.jpg`, and the download URL is saved on the expense. Expenses without a receipt save instantly offline; those with a receipt wait for the upload and show "UPLOADING…".

SECTION 12

State management (Riverpod)

All data flows from Firestore streams through Riverpod providers into widgets. Every stream provider watches `currentUidProvider`, so signing out tears down all listeners and signing in rebuilds them cleanly.

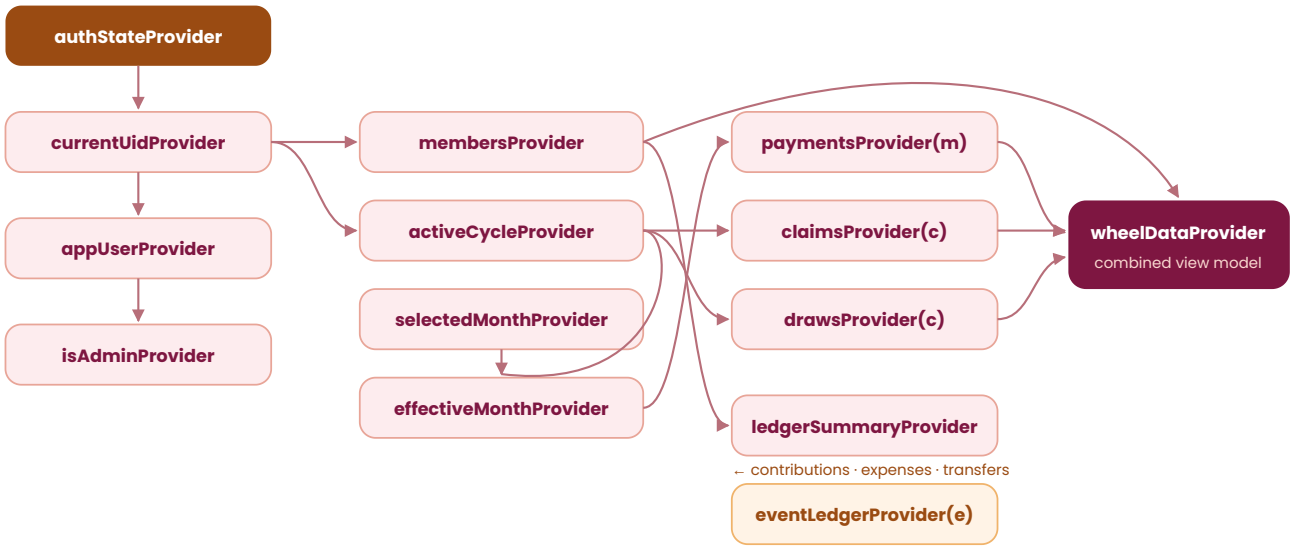


Figure 7 — Provider graph. `(m)` month, `(c)` cycle, `(e)` event = family parameters.

Provider	Type	Purpose
`authStateProvider`	StreamProvider<User?>	Firebase sign-in state.
`appUserProvider`	StreamProvider<AppUser?>	Live `users/{uid}` profile, role and member link.
`membersProvider`	StreamProvider<List<Member>>	All members ordered by seat number.
`activeCycleProvider`	StreamProvider<KuriCycle?>	The single active cycle (or null).
`selectedMonthProvider` → `effectiveMonthProvider`	StateProvider → Provider	User-picked month, clamped into the cycle range. Shared by Ledger and Pamparam tabs.
`paymentsProvider`	StreamProvider.family	Map memberId → Payment for one cycle-month.
`wheelDataProvider`	Provider<AsyncValue<WheelData?>>	Combines cycle, members, payments, claims, draws via `combineAsync` into eligibility, pot and this month's draw.
`eventLedgerProvider`	Provider.family	Combines the three wallet streams and runs `EventLedger.compute`.
`themeModeProvider`	StateProvider<ThemeMode>	Light / dark toggle from the profile menu.

`combineAsync()` merges several `AsyncValue`s: first error wins, loading until all have data — so screens show one spinner or one error instead of flickering.

SECTION 13

Security model

Firestore Security Rules (126 lines) and Storage Rules are the backend. Everything not explicitly allowed is denied.

Helper functions

```
function isAdmin()  { return hasProfile() && get(userPath()).data.role == 'admin'; }
function isMember() { return hasProfile(); }      // any registered user
function authPhone(){ return request.auth.token.get('phone_number', null); }
function validMoney(v) { return v is number && v >= 0 && v < 100000000; }
function monthKey(v)   { return v.matches('^[0-9]{4}-(0[1-9]|1[0-2])$'); }
```

Rules by collection

Path	Read	Write
`users/{uid}`	Self or admin	**Create** by self only as `member`, only if verified phone equals the linked member's phone, fixed field set. **Update/delete** admin only.
`members/{id}`	Members; unregistered user only her own phone's record	Admin; name ≥ 2 chars, phone E.164 regex, int seat, bool active.
`cycles/{id}`	Members	Admin; valid start month, 1–120 months, valid money, status active/closed. Never deleted.
`…/payments/{id}`	Members	Admin; ID must equal `monthKey_memberId`; bool paid; valid amount.
`…/draws/{month}` 🔒	Members	**Create only**, admin; winner's payment for that month must be `paid == true`; matching claim must exist after the same write. Update/delete: never.
`…/claims/{memberId}` 🔒	Members	**Create only**, admin; `hasClaimed == true`; the draw for `claimedMonth` must name this member. Update/delete: never.
`events/{id}` + entries	Members	Admin; title required; entries only in contributions/expenses/transfers with amount > 0.
Storage receipts	Signed-in	Admin (role looked up in Firestore); images only, < 8 MB.

Lucky-draw integrity guarantees

1. **One draw per month** — draw document ID is the month; a second write is an update, which is forbidden.
2. **Never win twice** — claim document ID is the member; it can be created once and never removed.
3. **Must be paid** — the rule reads the winner's payment record server-side.
4. **Draw and claim are inseparable** — `existsAfter / getAfter` require both in the same transaction.
5. **No re-rolls** — the result is committed before the animation starts.

Other protections

- All traffic is TLS-encrypted; Firebase stores data encrypted at rest.
- OTP abuse protection by Firebase (Play Integrity / APNs / reCAPTCHA fallback).
- Member phone numbers are visible only to signed-in, registered group members.

SECTION 14

Offline-first & sync

Firestore persistence is enabled in `main.dart` with an unlimited cache, so every screen opens instantly from the phone's local copy and keeps working without signal.

Operation	Offline?	Behaviour
View ledger, history, wallet, winners	Yes	Served from cache; refreshes automatically when online.
Toggle Paid / Unpaid	Yes	Applied locally at once; queued and synced later.
Add / edit member, start cycle	Yes	Sheet closes immediately; error snackbar if the server later rejects.
Add contribution / handover / expense without receipt	Yes	Same fire-and-forget pattern.
Expense with receipt photo	No	Upload needs network; shows UPLOADING… and an error if it fails.
Spin the wheel	No	Transactions need the server: “Draw failed (are you online?)”. Intentional — the draw must be verified.
First sign-in / OTP	No	Needs SMS and network. Existing sessions persist offline.

Conflict handling. Payment records are keyed per member per month and written as whole documents, so the last write wins. Because only admins write and each payment has one owner, conflicts are rare and harmless (the latest toggle is the truth). `markedBy` records which admin made the change.

SECTION 15

Screens & navigation

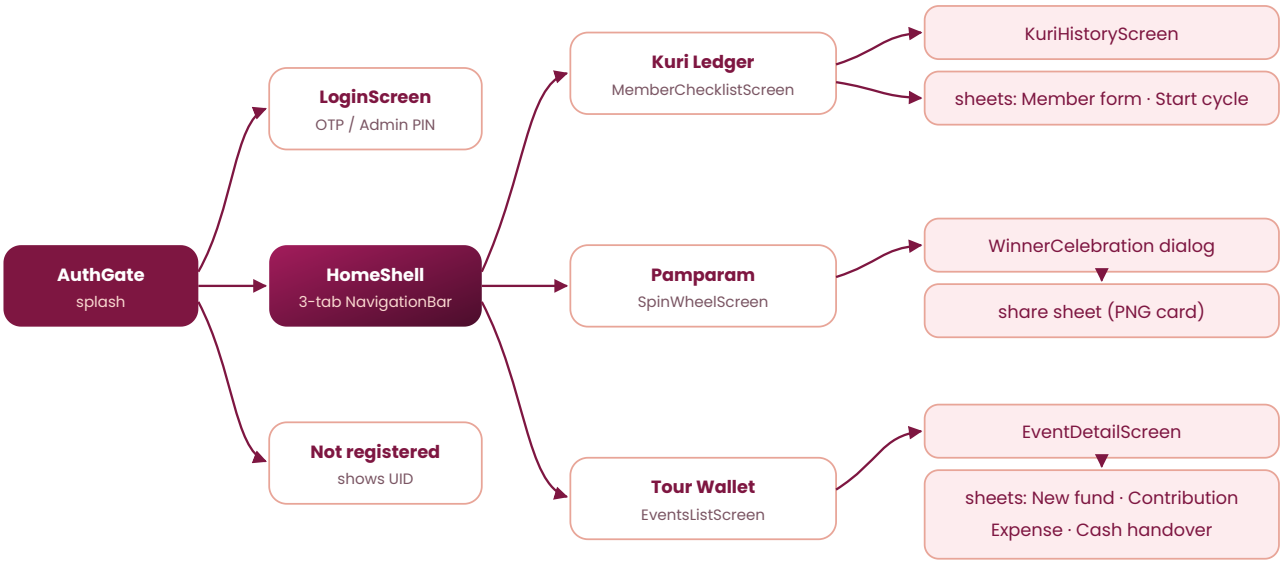


Figure 8 — Navigation map. All three tabs share the profile menu (dark mode, sign out).

Screen	Purpose	Notable UI
LoginScreen	Sign-in	Berry gradient, 3D badge, CheraSakhi script, segmented OTP / PIN tabs, gold button.
MemberChecklistScreen	Monthly ledger	Floating app bar, month switcher, bento metrics, search, filter chips, fixed-height 78 px rows in `SliverFixedExtentList` for smooth 60-item scrolling.
KuriHistoryScreen	Past months	Year chips, month chips with trophy markers, paid/unpaid name pills, winners timeline.
SpinWheelScreen	Lucky draw	Pot card with eligibility chips, glowing wheel, centre badge with slice count, eligibility rules panel, winners list.
EventsListScreen	All funds	Berry cards with emoji, collected / spent / balance.
EventDetailScreen	One fund	Collapsing header, rose-gold balance hero, holder board, category bars, timelines, “Add entry” FAB.

SECTION 16

Design system & brand



Logo

The group's own emblem — five women forming a flower, "CHERANMOOLA", "EST 2026" — rebuilt as a premium 3D badge: glossy jewel-tone figures (blue, amber, pink, red, green), pearl centre, rose-gold metal bezel on berry velvet, "LADIES WING" on the band and *CheraSakhi* in script.

Assets: `assets/brand/logo.png` (in-app), `app_icon.png`, `app_icon_foreground.png` (Android adaptive, background #7E1641). Regenerated by `docs/logo_build.py`.

Colour palette

Berry **#831843 · primary**	**Berry mid** **#A21C5B · gradients**	**Blossom** **#EC4899 · paid state**	**Rose gold** **#B76E79 · accent**
Rose light **#E8A598 · rings**	**Rose pale** **#F9D7CF · text on berry**	**Milk** **#FFF7F9 · light bg**	**Night** **#1C1020 · dark bg**

Typography

Playfair Display

Display, headline and title styles — elegant serif.

Poppins

Body, labels and captions — clean and very readable.

Components

Component	Description
`GlassCard`	Frosted acrylic surface: 85 % opacity, 16 px backdrop blur, hairline border, soft berry shadow. `blur:false` inside long lists for performance.
`GoldButton`	Rose-gold gradient call-to-action with glow, loading spinner and disabled state.
`AmbientBackground`	Soft blossom and rose-gold light blobs behind content so glass surfaces glow.

`MemberAvatar`, `EmptyState`, `SectionHeader`	Initials avatar (berry when paid), illustrated empty screens, consistent section titles.
Dark mode	Full deep-plum theme (#1C1020 / #2A1830), toggled from the profile menu.

SECTION 17

Integrations

Integration	How	Configuration
WhatsApp	`https://wa.me/<digits>?text=<encoded>` opened externally; no number → chat picker for group posts.	None. Phone stored in E.164.
Share sheet	Winner card captured as PNG to temp dir, shared with caption.	None.
Camera / gallery	`image_picker`, quality 70, max width 1600.	iOS usage strings in Info.plist.
Haptics & sound	Selection clicks on taps and wheel ticks; WAV assets in `assets/sounds/`.	Bundled.
App icon	`flutter_launcher_icons` from brand assets.	`dart run flutter_launcher_icons`

SECTION 18

Testing strategy

Business rules live in pure Dart, so they are covered by fast unit tests that need no Firebase.

Test (test/widget_test.dart)	Verifies
Eligibility — paid & not-won	Only paid, active, not-yet-won members are eligible; unpaid and inactive excluded; previous winner listed as “already won”.
Eligibility — previous winner	A paid member who already won stays excluded.
Ledger summary	3 members, 2 paid at ₹1,000 → paid 2, target ₹3,000, collected ₹2,000, deficit ₹1,000.
Cash-holder balances	The worked example in Section 11: H = ₹800, K = -₹100, fund = ₹700.

Recommended additions before launch

- **Security-rules tests** with the Firebase Emulator Suite (`@firebase/rules-unit-testing`): member cannot write; second draw rejected; unpaid winner rejected; claim cannot be deleted.
- **Widget tests** for the ledger toggle and filter chips with a fake repository.
- **Manual acceptance** on one low-end Android phone with 60 seeded members: scroll smoothness, spin at 60 names, offline toggle then reconnect.

SECTION 19

Setup, build & deployment



Step by step

```
# 1. Generate Android/iOS folders (keeps lib/ untouched)
cd kuri_collective
flutter create . --org com.cherasakhi --platforms android,ios
flutter pub get
dart run flutter_launcher_icons

# 2. Connect Firebase (Auth: enable Phone + Email/Password; create Firestore + Storage)
dart pub global activate flutterfire_cli
flutterfire configure            # replaces lib/firebase_options.dart

# 3. Deploy security rules
npm i -g firebase-tools && firebase login
firebase use <project-id>
firebase deploy --only firestore:rules,storage

# 5. Test & build
flutter test
flutter build apk --release          # share directly with members
flutter build appbundle --release    # Google Play
flutter build ipa                    # App Store (Mac + Xcode)
```

Platform configuration checklist

Item	Where	Value
App name	AndroidManifest.xml / Info.plist	`android:label="CheraSakhi"` / `CFBundleDisplayName`
Android min SDK	android/app/build.gradle	`minSdk = 23`
Phone auth (Android)	Firebase console → Android app	SHA-1 + SHA-256 (`./gradlew signingReport`); enable Play Integrity API
Phone auth (iOS)	Firebase → Cloud Messaging; Xcode URL types	APNs key; `REVERSED_CLIENT_ID` URL scheme
iOS target	Podfile	`platform :ios, '13.0'`
Camera / photos	Info.plist	`NSCameraUsageDescription` , `NSPhotoLibraryUsageDescription`
Firestore region	Firebase console (one-time, permanent)	`asia-south1` (Mumbai) for lowest latency in Kerala

Hosting cost estimate

For 60 members the app sits comfortably inside the free Firebase Spark plan for Firestore and Storage (50,000 reads/day, 20,000 writes/day, 5 GB storage). Phone-auth SMS is billed per verification by Google once the free allowance is used; members stay signed in, so SMS are mostly needed only on first install. Check current Firebase pricing before launch, as quotas change.

SECTION 20

Operations runbook

One-time: create the first admin

1. Firebase console → Authentication → **Add user**: e.g. `admin@cherasakhi.app`, password = a 6+ digit PIN.
2. Copy the new user's **UID**.
3. Firestore → create document `users/{UID}` with `{ "name": "President", "role": "admin" }`.
4. In the app choose **Admin PIN** and sign in.
5. To promote a member: after her first OTP login, edit her `users/{uid}` and set `role` to `admin`.

Monthly cycle

When	Who	Action in app
Day 1–10	Admin	Kuri Ledger → mark each payment as it arrives.
Day 8	Admin	Filter **Unpaid** → "Remind all unpaid" → post in WhatsApp group; or remind individually.
Meeting day	Admin	Pamparam tab → check the eligibility panel aloud → **Spin the wheel** (screen-mirror to a TV for the group).
After draw	Admin	Share the winner card to the group; hand over the pot.
Any time	Members	Check their status, history and winners on their own phones.

New member joins / member leaves

- **Joins:** Kuri Ledger → ⊕ → Add member (name + WhatsApp number). She installs the app and logs in with OTP.
- **Leaves:** long-press → turn off **Active member**. History is kept; she is excluded from totals and future draws.

Trip or celebration

1. Tour Wallet → New fund (category, title, per-head amount).
2. Record each contribution, choosing who received the cash.
3. During the trip, log expenses with receipt photos and record handovers.
4. Settle up using the holder board: green holders return cash, red holders are reimbursed.

Troubleshooting

Symptom	Cause → Fix
"Not registered yet" after OTP	Her phone number in Members differs (missing +91, typo) → admin edits the member's number, she signs out and in.
OTP never arrives (Android)	SHA fingerprints missing in Firebase or Play Integrity not enabled.
"Draw failed (are you online?)"	No network — move to Wi-Fi and spin again (nothing was saved).
"The draw for this month is already completed."	Expected — the month is locked; open the Draw completed card to re-share.

Permission-denied snackbars	User isn't admin, or rules weren't deployed → run `firebase deploy --only firestore:rules,storage`.

SECTION 21

Limitations & roadmap

Known limitations (honest assessment)

Limitation	Impact	Mitigation
Code not yet compiled in a Flutter SDK	The package was written and import-checked but not built; a first `flutter analyze` may show small fixes.	Run analyze + build once and fix any reported lines.
Random winner chosen on the admin's phone	Rules guarantee *who can* win, not *how* the winner was picked. A modified app could choose any eligible member.	Roadmap: move selection to a Cloud Function using server randomness.
No edit history for payments	A toggle overwrites the previous state; only the last admin is recorded.	Roadmap: append-only audit log.
Wallet entries can't be edited in the UI	Mistakes need a correcting entry or console edit.	Roadmap: edit / reverse entry with reason.
`isSettled` flag not exposed	Funds can't be marked closed from the app yet.	Roadmap: “Settle fund” action.
Draw needs internet	Cannot spin offline.	By design — fairness requires the server check.
Admins created via console	Requires a technical person once.	Roadmap: “Promote to admin” in app for existing admins.
English UI	Some members may prefer Malayalam.	Roadmap: `flutter_localizations` + Malayalam strings.

Roadmap

Phase 2 — Trust

- Server-side draw (Cloud Function)
- Audit log of every change
- Rules unit tests in CI

Phase 3 — Members

- Malayalam language
- Push notifications (FCM): reminders, live draw alerts
- Personal “My payments” page

Phase 4 — Admin

- PDF / Excel monthly statements
- UPI payment links
- Edit & settle wallet entries
- In-app admin management

APPENDIX A

Source file inventory

All Dart files in the delivered package with line counts.

File	Layer	Lines
`lib/app.dart`	App	23
`lib/core/constants/app_constants.dart`	Core	13
`lib/core/network/firestore_utils.dart`	Core	14
`lib/core/providers/firebase_providers.dart`	Core	17
`lib/core/theme/app_colors.dart`	Core	40
`lib/core/theme/app_theme.dart`	Core	106
`lib/core/theme/theme_controller.dart`	Core	4
`lib/core/utils/async_combine.dart`	Core	12
`lib/core/utils/formatters.dart`	Core	47
`lib/core/utils/month_key.dart`	Core	29
`lib/core/utils/whatsapp.dart`	Core	44
`lib/core/widgets/ambient_background.dart`	Core	46
`lib/core/widgets/empty_state.dart`	Core	52
`lib/core/widgets/glass_card.dart`	Core	81
`lib/core/widgets/gold_button.dart`	Core	79
`lib/core/widgets/member_avatar.dart`	Core	34
`lib/core/widgets/section_header.dart`	Core	31
`lib/features/auth/data/auth_repository.dart`	Data	90
`lib/features/auth/domain/app_user.dart`	Domain	21
`lib/features/auth/presentation/providers/auth_providers.dart`	Presentation	19
`lib/features/auth/presentation/screens/auth_gate.dart`	Presentation	145
`lib/features/auth/presentation/screens/login_screen.dart`	Presentation	241
`lib/features/auth/presentation/widgets/profile_menu_button.dart`	Presentation	74
`lib/features/events/data/firestore_events_repository.dart`	Data	187
`lib/features/events/domain/entities/event_models.dart`	Domain	220
`lib/features/events/domain/repositories/events_repository.dart`	Domain	51
`lib/features/events/presentation/providers/events_providers.dart`	Presentation	44

lib/features/events/presentation/screens/event_detail_screen.dart	Presentation	444
lib/features/events/presentation/screens/events_list_screen.dart	Presentation	171
lib/features/events/presentation/widgets/event_sheets.dart	Presentation	532
lib/features/events/presentation/widgets/expense_style.dart	Presentation	19
lib/features/events/presentation/widgets/member_dropdown.dart	Presentation	36
lib/features/home/presentation/home_shell.dart	Presentation	54
lib/features/kuri/data/firestore_kuri_repository.dart	Data	120
lib/features/kuri/data/kuri_mappers.dart	Data	54
lib/features/kuri/domain/entities/claim.dart	Domain	19
lib/features/kuri/domain/entities/kuri_cycle.dart	Domain	36
lib/features/kuri/domain/entities/ledger_summary.dart	Domain	45
lib/features/kuri/domain/entities/member.dart	Domain	41
lib/features/kuri/domain/entities/payment.dart	Domain	19
lib/features/kuri/domain/repositories/kuri_repository.dart	Domain	33
lib/features/kuri/presentation/providers/kuri_providers.dart	Presentation	60
lib/features/kuri/presentation/screens/kuri_history_screen.dart	Presentation	179
lib/features/kuri/presentation/screens/member_checklist_screen.dart	Presentation	332
lib/features/kuri/presentation/widgets/ledger_metrics_bento.dart	Presentation	173
lib/features/kuri/presentation/widgets/member_form_sheet.dart	Presentation	119
lib/features/kuri/presentation/widgets/member_payment_tile.dart	Presentation	115
lib/features/kuri/presentation/widgets/month_switcher.dart	Presentation	60
lib/features/kuri/presentation/widgets/start_cycle_sheet.dart	Presentation	141
lib/features/lottery_wheel/data/firestore_draw_repository.dart	Data	95
lib/features/lottery_wheel/domain/entities/draw_record.dart	Domain	22
lib/features/lottery_wheel/domain/repositories/draw_repository.dart	Domain	26
lib/features/lottery_wheel/domain/usecases/eligibility.dart	Domain	39
lib/features/lottery_wheel/presentation/providers/wheel_providers.dart	Presentation	75
lib/features/lottery_wheel/presentation/screens/spin_wheel_screen.dart	Presentation	530
lib/features/lottery_wheel/presentation/widgets/wheel_painter.dart	Presentation	159
lib/features/lottery_wheel/presentation/widgets/wheel_sfx.dart	Presentation	32
lib/features/lottery_wheel/presentation/widgets/winner_celebration.dart	Presentation	313

`lib/firebase_options.dart`	App	14
`lib/main.dart`	App	22
`test/widget_test.dart`	Test	62
`firestore.rules`	Security	126
`storage.rules`	Security	16
Total Dart (61 files)		**5,955**